

RAG Systems Design

Building knowledge-aware AI products — from architecture to deployment

What This Module Is About

Retrieval-Augmented Generation (RAG) is the architectural pattern that separates production AI systems from unreliable demos. Instead of relying on an LLM's training data, RAG grounds every response in your own documents, knowledge bases, and live data — making it factual, traceable, and updatable.

The 8-Stage RAG Pipeline

Stage	What Happens	Tools / Notes
1. Ingestion	Documents split into chunks	PyPDF2, LangChain splitters
2. Embedding	Chunks converted to vectors	OpenAI, Cohere, HuggingFace
3. Vector Storage	Vectors stored and indexed	pgvector, Pinecone, Weaviate
4. Similarity Search	Relevant chunks retrieved for query	cosine similarity, ANN search
5. Context Assembly	Top chunks assembled into prompt context	rank, deduplicate, trim
6. LLM Integration	Context + question sent to LLM	Claude, GPT-4, Mistral
7. Response Gen.	LLM generates grounded response	streaming or batch
8. Evaluation	Quality measured (precision, faithfulness)	RAGAS, custom metrics

Key Concepts

- **Chunking Strategy** — chunk too small and you lose context; too large and retrieval becomes noisy. 256–512 tokens with 10–20% overlap is a common starting point.
- **Embedding Model Choice** — the embedding model you use must match what you use at query time. Consistency is critical.
- **Vector Database** — for small projects use pgvector (Postgres). For large-scale production consider Pinecone or Weaviate.
- **Relevance Scoring** — always log the similarity scores returned. A score below ~0.7 often means no good match was found — handle this gracefully.
- **Evaluation First** — define how you will measure RAG quality (faithfulness, relevance, completeness) before you start building.

Tip: Build the simplest possible RAG pipeline first (text files → in-memory list → LLM). Only introduce a vector database once the basic flow works end-to-end.

Resources for Deeper Learning

1 LlamaIndex Documentation

<https://docs.llamaindex.ai/>

The most comprehensive Python framework for building RAG systems. Read the 'Understanding' section for deep conceptual grounding, then the Quickstart to see it in action.

2 pgvector — Vector Extension for PostgreSQL

<https://github.com/pgvector/pgvector>

If you already use Postgres, pgvector adds vector similarity search with no new infrastructure. Install it, create an embeddings table, and you have a production-ready vector store.

3 Pinecone Learning Center

<https://www.pinecone.io/learn/>

Pinecone's free guides on RAG, embeddings, semantic search, and vector databases. Well-written, practical, and largely framework-agnostic.

4 RAGAS — RAG Evaluation Framework

<https://docs.ragas.io/>

RAGAS provides automated metrics for RAG quality: faithfulness (does the answer follow from context?), answer relevancy, and context precision. Essential for systematic improvement.

5 Anthropic Cookbook — RAG Examples

<https://github.com/anthropics/anthropic-cookbook>

Official Anthropic examples showing how to use Claude for RAG, tool use, and multi-document reasoning. Excellent starting templates for your own system.

6 Weaviate Academy

<https://weaviate.io/developers/academy>

Free courses on vector search, RAG architectures, and hybrid search (vector + keyword). Concept explanations are useful even if you choose a different vector database.

Module 3 Assignment — Build a Basic RAG Pipeline

- Choose a small document set (5–10 PDFs or text files on any topic you like). Write a script to read and chunk them into 300-token segments with 50-token overlap.
- Generate embeddings for each chunk using an embedding API (OpenAI, Cohere, or HuggingFace). Store them in pgvector or a simple JSON file for now.
- Build a query function: take a user question → embed it → retrieve top-3 most similar chunks → pass them as context to Claude → return the answer.
- Test with 5 questions. For each one, log: the question, retrieved chunks, and the final answer. Are the retrieved chunks actually relevant?
- Write a brief reflection (in your README): what worked, what failed, and one thing you would improve.

Note: This pipeline is the foundation of your capstone. Keep the code clean and modular.